

Do something today
that your future self
will thank you for.



Good

Evening

Today's content

01. Introduction to Queues
02. Implementation of Queues
 - using Arrays
 - using LinkedList
 - using stacks
03. Generate K^{th} no. using 1 & 2
04. Max of every window (Deque)

Sliding Window Maximum (v v v I)

Queue → linear data structure where elements get inserted from one end & get removed from the other end.

→ First In First Out (principle)



* Some real world Examples

01. Message queue
02. Customer service
03. Ticket counter
04. Printer
05. Toll plaza

* Common operation in Queue → TC: $O(1)$

01. Enqueue → Add an element inside the queue. ($q.add(x)$)
02. Dequeue() → Remove the first element & return it. ($q.remove()$)
03. Peek() → peek() operation allows you to look at the

first element in queue. ($q.peek()$)

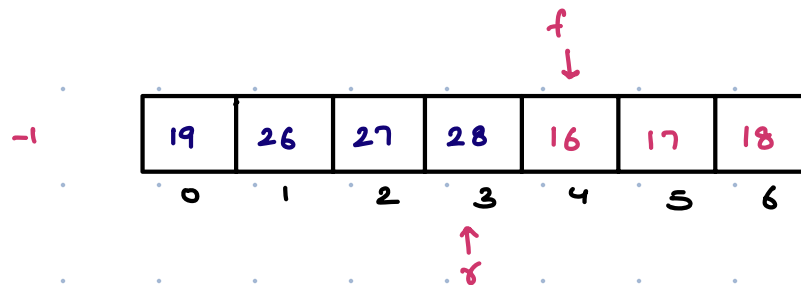
04. $size()$ → return the no. of element present at queue. ($q.size()$)

* Implementation of Queue

01. Array

$f = -1$ → idx of last ele which just got removed

$r = -1$ → idx of last ele which just got inserted



Add(5)	remove()	Add(16)	Add(26)	peek()
Add(6)	remove()	Add(17)	Add(27) ←	peek()
Add(7)	remove()	Add(18)	Add(28) ←	
Add(9)	remove();	Add(19)	remove() → 16	

if ($f == r$) → Queue is empty

└→ Queue is full



void add(x) {

if ($sz == n$) return;

$r = (r + 1) \% n$

$A[r] = x;$

$sz++;$

int size() {

return sz ;

}

```
int remove() {
```

```
    if (sz == 0) return -1;
```

```
    f = (f + 1) % n
```

```
    int val = A[f]
```

```
    sz--;
```

```
    return val;
```

```
}
```

```
int peek()
```

```
    if (sz == 0) return -1;
```

```
    int idx = (f + 1) % N
```

```
    return A[idx];
```

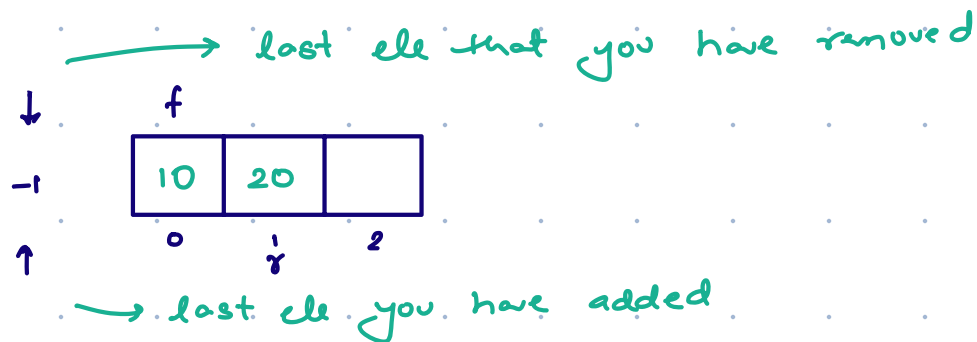
```
}
```

```
boolean isEmpty()
```

```
    if (sz == 0) return true;
```

```
    else return false;
```

```
}
```



```
Add(10)
```

```
peek() → 10
```

```
Add(20)
```

```
remove()
```

* Implement Queue using Linkedlist

↳ FIFO

Add(5) ✓

Add(6) ✓

Add(7) ✓

remove()

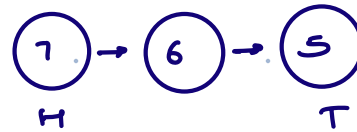
remove()

Add(8)

X

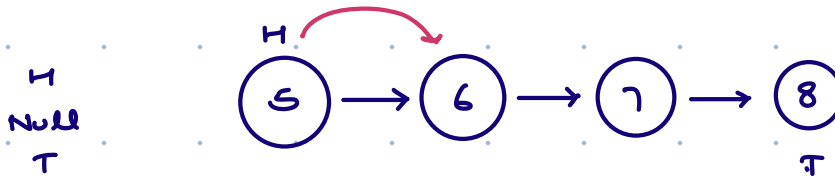
* Insert at head $\rightarrow O(1)$

* Remove at tail $\rightarrow O(n)$



* Insert at Tail $\rightarrow O(1)$

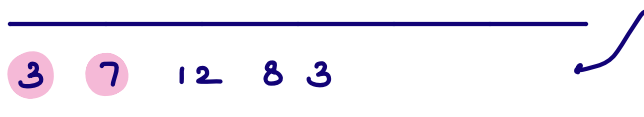
Remove at Head $\rightarrow O(1)$



Tail.next = new Node(8)

Tail = Tail.next

What will be the state of the queue after these operations enqueue(3), enqueue(7), enqueue(12), dequeue(), dequeue(), enqueue(8), enqueue(3)



Output = 12 8 3

What will be the state of the queue after these operations enqueue(4), dequeue(), enqueue(9), enqueue(3), enqueue(7), enqueue(11), enqueue(20), dequeue()

4 9 3 7 11 20

Output = 3 7 11 20

* Implementation using stacks

front 5 4 7 8 9 rear

01. Add(5) → st.push(5)

02. Add(4) → st.push(4)

03. Add(7) →

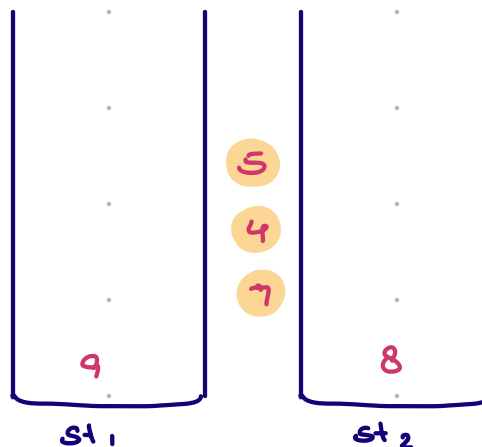
04. Add(8) →

05. remove() ←

06. remove()

07. Add(9)

08. remove()



Add(x) → st₁.push(x) → Tc: O(1)

Remove() → if (st₂.size() == 0) → Tc: O(1)

Yes No

Move all ele
from st₁ to st₂

st₂.pop();

→ st₂.pop();

size() → st₁.size() + st₂.size();

01. Add(5)

02. Add(4)

03. Add(7)

04. Add(8)

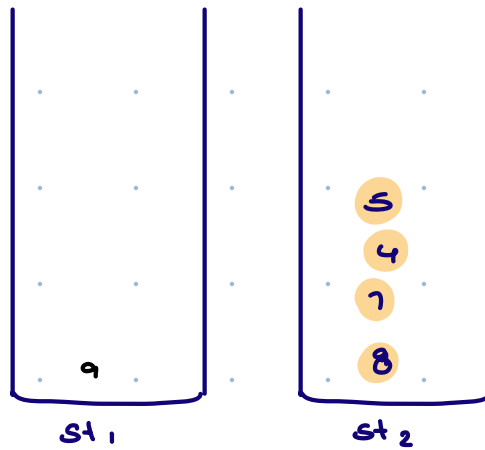
05. remove() ✓

06. Add(9)

07. remove()

08. remove()

09. remove()



1st rem = Move all ele from st_1 to st_2
& $st_2.pop()$

8ops + 1ops

2nd rem = $st_2.pop()$ = 1ops

3rd rem = $st_2.pop()$ = 1ops

4th rem = $st_2.pop()$ = 1ops

4 rem = 12 ops

1 rem = $\frac{12}{4} \approx 3 \text{ ops} \approx O(1)$

10:33 pm → 10:43 pm

N^* perfect no.

Generate K^* perfect no. in series using only 1 & 2 as digits.

Note $\rightarrow$ The series should be in increasing order

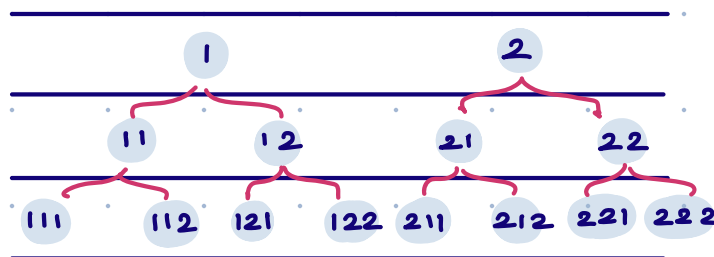
	1	2	11	12	21	22	111	112	121	122
K =	1	2	3	4	5	6	7	8	9	10

Brute force $\rightarrow$ Start iterating from 1 & keep increasing the count whenever we get the number that is only made up of 1, 2 or its combination.

if (count == K) return no.

Idea 2 Data in form of levels / data in digits is following some pattern

1 digit
2 digit



1	2	11	12	21	22	111	112	121	122
---	---	----	----	----	----	-----	-----	-----	-----

↓
Ans

K = 5


```
int solve (N)
```

```
if (N ≤ 2) return N
```

```
Queue <Integers> q ;
```

```
q.add(1)
```

```
q.add(2)
```

```
i = 3
```

```
while (i ≤ N) {
```

```
    int rem = q.remove();
```

```
    a = rem * 10 + 1
```

```
    b = rem * 10 + 2
```

```
    if (i == N) return a;
```

```
    if (i+1 == N) return b;
```

```
    q.add(a)
```

```
    q.add(b)
```

```
    i = i + 2 ✓
```

```
}
```

```
}
```

N = 1 Ans = 1

N = 2 Ans = 2

N = 7

~~9~~ = ~~3~~ ~~3~~ 7

queue

1 2 11 12 21 22

a = 111

b = 112

Tc: $O(n)$

Sc: $O(n)$

* Doubly Ended queue (Deque)

→ Data structure where we can go & access elements from both sides.

→ Add elements from front & last

→ Remove elements from front & last.

* Deque can act both as stack & Queue.

Functionalities

01. Addfirst()	} TC: O(1) {	04. Addatlast()
02. Removefirst()		05. Remove last()
03. getfirst()		06. getlast()

Stack

* Sliding Window Maximum

Q Given an integer array A & a window of size k.
Find the maximum of all windows.

$A[] = \{10, 1, 9, 3, 7, 6, 5, 11, 8\}$ $k = 4$

$Ans[] = \{10, 9, 9, 7, 11, 11\}$

$A[] = \{1, 4, 3, 2, 5\}$ $k = 3$

$Ans[] = \{4, 4, 5\}$

Brute force $\rightarrow$ Generate all subarrays of size k & iterate on the subarray to get the maximum.

$$TC: O(n-k+1) * k \approx \underbrace{O(n^2)}_{k = n/2}$$

$A[] = \{ 3, 15, 6, 12, 4, 2, 10, 9, 13, 2 \}$

~~3~~ ~~15~~ ~~6~~ ~~12~~ ~~4~~ ~~2~~ ~~10~~ ~~9~~ 13 2

Ans = 15 15 12 12 10 13 13

01. Remove from last → help us to maintain only probable maximum in DS
02. Access the front ele
03. Remove ele from front → help you in dropping previous window ele
04. Add ele from last

void maxwindow (int [] A, int k)

```
Deque<I> dq = new ArrayDeque();
```

```
// Insert first window
```

```
for (i=0; i<k; i++) {
```

```
    while (dq.size() > 0 && A[i] > dq.getLast())
```

```
        dq.removeLast();
```

```
    }
```

```
    dq.addLast(A[i]);
```

```
}
```

```
print ( dq.getFirst());
```

```
// For all other window
```

```
s = 1
```

```
e = k
```

$Tc: O(N)$

$Sc: O(N)$

```
while ( e < n ) {
```

```
    // remove A[s-1] if present
```

```
    if ( A[s-1] == dq.getFirst() ) {
```

```
        dq.removeFirst();
```

```
    }
```

```
    // add A[e]
```

```
    while ( dq.size() > 0 && A[e] > dq.getLast() )
```

```
        dq.removeLast();
```

```
    }
```

```
    dq.addLast ( A[e] );
```

```
    print ( dq.getFirst());
```

```
    s++;
```

```
    e++;
```

```
}
```